

Natural Language Task Specification for Autonomous UAV Detection and Tracking

Nathaniel DeLeon *

Joanna Gutierrez †

Jorge Rios ‡

Gael Marquez §

Efren Saenz ¶

Abstract

Autonomous UAV systems traditionally rely on pre-trained, narrow-vocabulary models that require rigid target specifications, limiting their operational flexibility. This paper presents a pipeline for natural language task specification for autonomous drone detection and tracking that replaces fixed object parameters with open-ended human intent. Our dual-model architecture integrates a Large Language Model (LLM) for semantic reasoning with YOLOv8m-World open-vocabulary detection, offloading computation to a ground station, and communicating flight commands via the custom MAVLink interface/GCS (ground control station) over UDP. A boustrophedon search pattern governs initial area coverage, transitioning to proportional visual tracking upon target confirmation. Evaluated across six prompt categories of increasing semantic complexity in simulation, the system achieved a 95% correct detection rate in AI mode compared to 30% using YOLOv8m-World alone, with failures in standard mode concentrated on color disambiguation, intent, and relational descriptions. These results establish the LLM as a necessary component for intent-driven UAV missions rather than relying solely on the YOLOv8m-World model.

1 Introduction

Autonomous Drones exist as a subclass within the UAV field, and therefore, have set requirements such as object detection. Object detection is a necessity for any autonomous unmanned vehicle to operate safely. Traditionally, object detection in UAVs is accomplished by defining a specified object to search for and letting the onboard computer identify said object. Through the semester, we have developed a project with the purpose of expanding on this idea, rather than focusing on defining one type of object, we use a dual model architecture to allow for the use of natural language. Our goal is to be able to detect objects unknown to the system beforehand, without prior training.

*Department of Computer Science, University of Texas Rio Grande Valley, nathaniel.deleon02@utrgv.edu

†Department of Computer Science, University of Texas Rio Grande Valley, joanna.gutierrez03@utrgv.edu

‡Department of Computer Science, University of Texas Rio Grande Valley, jorge.rios03@utrgv.edu

§Department of Computer Science, University of Texas Rio Grande Valley, gael.marquez01@utrgv.edu

¶Department of Computer Science, University of Texas Rio Grande Valley, efren.saenz01@utrgv.edu

2 Related Work

Our project ventures into a rapidly growing subfield of UAV robotics, evolving from existing object detection pipelines toward a novel integration of Large Language Models (LLMs) to expand semantic understanding. Current architectures, such as LPANet, leverage semantic features extracted from LLMs to guide progressive spatial alignment for multimodal UAV object detection [11]. By using fine-grained, LLM-generated text descriptions of object categories, the system achieves significantly higher accuracy from aerial viewpoints than traditional pre-trained models. Following the initial detection phase, recent frameworks utilize LLMs as high-level reasoning engines to process raw visual outputs. Rather than displaying static bounding boxes, these pipelines feed extracted data into the LLM to identify semantic anomalies within the environment. By framing visual data through a continuous stream of prompts, the LLM acts as an active interpreter, translating raw detection data into structured, context-aware evaluations of the scene. Despite these cognitive gains, running an LLM alongside a heavy vision model introduces a massive computational burden. This simultaneous processing often causes significant latency, creating a critical bottleneck for real-time drone control [10]. A primary hurdle with LLM integration lies in the sequential nature of token generation, which can act as a bottleneck for responsiveness.

To address this, recent literature suggests bypassing verbose programming languages in favor of MiniSpec, a novel, token-efficient language [6]. By employing stream interpretation, this architecture minimizes plan generation delays and reduces response times by up to 62%, proving that efficient command transmission is as vital as the processing itself. Beyond latency, autonomous drones must exhibit adaptability in unstructured environments. As illustrated in recent work on aerial additive manufacturing, traditional drone-based construction is often hindered by flight instability and rigid planning [9]. By integrating LLMs, these systems can dynamically generate and adapt building plans on-site through semantic reasoning. This feedback-based approach allows the system to correct unforeseen physical errors automatically, converting simple text prompts into physical structures with 90% build accuracy.

3 Background

The evolution of Unmanned Aerial Vehicles (UAVs) has seen rapid progression across various levels of autonomy, ranging from basic pilot assistance to fully autonomous,

human-out-of-the-loop systems. A catalyst of this transition is the integration of Computer Vision and Artificial Intelligence directly into the drone’s operational system. Typically, these models operate entirely onboard the vehicle’s companion computer. This allows for real time decision-making without the latency or unreliability associated with continuous human connection over a network link. Within more defined applications such as Search and Rescue, the capability of a drone to recognize a specific object and subsequently execute a certain behavior is necessary [7]. However, an existing limitation in current autonomous systems is the need for strict target specification. Traditional computer vision pipelines require pre-trained, narrow-vocabulary models or strictly defined digital parameters, severely limiting operational flexibility. The objective of this project is to address this issue by allowing for a highly generalizable user experience through a custom user interface. The interface would be capable of handling significantly more complex searching tasks through the use of natural language - unstructured, human-like communication that natively conveys intent, context, and relationships. Natural language can then fundamentally shift how operators interact with autonomous UAVs, allowing for intent-driven missions rather than structured searching.

4 Technologies

The following frameworks and models form the technical foundation of the system.

4.1 ArduPilot Firmware

At the foundation of the drone and simulations physical flight mechanics is ArduPilot, a highly reliable, open-source flight controller firmware [1]. ArduPilot governs the fundamental aerodynamics, motor mixing, and lower-level autonomous behaviors of the UAV. It provides the essential interface, allowing the companion computer to issue automated GUIDED mode waypoints and dynamic positional overrides during active tracking.

4.2 YOLOv8m-World

The YOLOv8m-World model serves as the core Computer Vision model for this architecture. Unlike traditional convolutional neural networks that are limited to a predefined set of detectable classes, YOLOv8m-World utilizes real-time open-vocabulary detection [8]. Allowing the system to identify any object specified by the set text prompts.

4.3 MAVLink

MAVLink is a protocol designed for communication between unmanned vehicles and ground control stations [3]. Operating over serial or UDP connections, MAVLink encodes data into compact binary packets, making it well-

suited for internet-constrained environments. It also provides a virtual simulator (SITL), allowing us to safely test our flight code in a simulated physics environment before using a physical drone.

4.4 Robot Operating System 2 (ROS 2)

ROS 2 is a widely adopted open-source robotics middleware framework that provides standardized communication through a publish/subscribe messaging model. Computation is organized into *nodes*, each responsible for a specific task, which communicate through named *topics* [4]. This architecture allows individual components such as the camera bridge, search planner, and flight controller interface to be developed and tested independently before integration. In our system, ROS 2 serves as the communication backbone on the companion computer, carrying camera frames, target prompts, detection flags, and flight setpoints between nodes. MAVROS, a ROS 2 package, further abstracts the MAVLink interface by exposing ArduPilot commands and telemetry as standard ROS topics and services.

4.5 Gazebo Harmonic

Gazebo Harmonic is a physics-based robotics simulator that provides models to build set environments [2]. It supports SDF (Simulation Description Format) world definitions, allowing developers to construct environments populated with static and dynamic models. For this project, Gazebo simulates the drone’s camera feed, physics, and environment, providing a controlled testbed for the full detection and navigation pipeline. The `ros_gz_bridge` package bridges Gazebo’s internal topic system with ROS 2, making simulated sensor data available to ROS nodes in the same format as real hardware.

5 Methods

The project consisted of three core branches of development: the ground station, the drone, and the integration of both. All development and validation was conducted in simulation using ArduPilot’s SITL firmware alongside Gazebo Harmonic, enabling safe and rapid iteration without requiring a physical drone.

5.1 Drone Side Development

The drone side of the system is responsible for autonomous flight execution and serves as the communication endpoint for ground station commands. It runs entirely on the companion computer aboard the drone and is composed of two ROS2 nodes: a UDP relay node and a boustrophedon flight node.

5.1.1 UDP Relay Node

The UDP relay node acts as the bridge between the drone’s onboard systems and the ground station. On

startup, the node broadcasts a discovery beacon over UDP to locate the ground station on the local network. Once a connection is established, the node subscribes to the onboard camera topic and continuously streams compressed JPEG frames to the ground station at 640×480 resolution. The relay simultaneously listens on a dedicated command port for instructions returned by the ground station. It handles four primary command types: **TRACKING**, **FOUND**, **RTL**, and **IDLE**. Upon receiving a **TRACKING** command, the relay spawns the boustrophedon search process, initiating the drone’s autonomous area coverage. **FOUND** commands carry the normalized center of the target’s bounding box, forwarded to the lawnmower node via the `/target_offset` topic so that the drone is able to steer toward the object. The **RTL** command terminates any active mission and switches the drone to return-to-launch mode via the `MAVROS /mavros/set_mode` service.

5.1.2 Boustrophedon Search Node

When a tracking command is received, the system initiates an autonomous search using a boustrophedon, or “lawnmower”, flight pattern. This pattern sweeps the drone across a predefined 12×12 meter area in alternating horizontal lanes spaced 2 meters apart at a cruise altitude of 2 meters, providing systematic coverage of the search zone.

The node arms the drone, switches to **GUIDED** mode, and commands takeoff before beginning the pattern. Each waypoint is published at 20 Hz to the `/mavros/setpoint_position/local` topic, and the drone advances to the next waypoint once it comes within a 0.5 meter acceptance radius. The pattern loop continuously checks a `tracking_active` flag; when the ground station reports a detection via the `/object_found` topic, the loop exits immediately.

Upon detection, the node transitions from search to approach. A proportional controller reads normalized X/Y offsets published by the ground station and incrementally steers the drone toward the target at 20 Hz. An exponential moving average ($\alpha = 0.3$) is applied to the offset signal to suppress noise, and a dead zone threshold prevents oscillation once the target is centered in frame. The drone then holds its final position until an **RTL** command is received.

5.2 Ground Station Development

The ground station acted as the foundation of the project itself. Rather than relying on a traditional, cluttered dashboard of manual flight controls, we designed the interface to be intent-driven. The incorporation of a User Interface that utilizes natural language processing acted as the brain of the entire system, bridging human communication with complex computer vision.

5.2.1 User Interface

The user interface (UI) was designed to be simple. Since the project was heavily reliant on python for other components, we decided to continue working within that ecosystem for consistency. We utilized the PyQt6 library to design the UI.

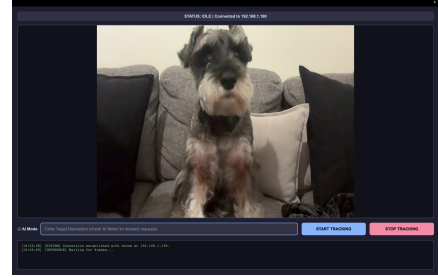


Figure 1: Functional UI

The UI provides clear labels for the status and connected IP to allow for the user to know both the connected device as well as the current status of the drone (**IDLE**, **TRACKING**, **FOUND**, **LOST**). Additionally we implemented buttons that allow the user to submit their tracking item, to stop tracking, or change the tracking type to ‘AI Mode’. However, the highlight of the UI would be focused on two core components: the streaming video feed and the log terminal. Annotated Video Feed is displayed within the center of the UI design, and what is happening within the ground station system is logged on the ground terminal. These allow for the user to have constant understanding of what is happening within the system, whether that be visual (what the drone is seeing) or logical (behind the scenes system execution).

5.2.2 Detection Pipeline

The detection pipeline is built upon a dual-model architecture, leveraging the capabilities of GPT-5.2 and YOLOv8m-World. Operating in tandem, GPT-5.2 performs high-level semantic reasoning to define initial targets. When the start tracking button is pressed, an API call to GPT 5.2 is made every two seconds, where both the current frame and input prompt is given. GPT 5.2 then utilizes both its semantic and spatial reasoning to detect if an object matching that description is within the current frame. Upon receiving a positive confirmation response(it found the object), the pipeline halts further API calls to GPT-5.2, terminating the background semantic search to conserve computational resources and minimize network latency. The pipeline is then fully transitioned to a continuous, real-time tracking phase driven by YOLOv8m-World. The bounding box coordinates provided by the LLM are dynamically scaled to match the drone’s live video feed, establishing a continuous lock for YOLOv8m-World.

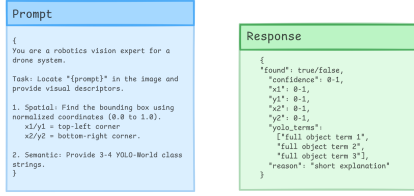


Figure 2: LLM API Call and Response

5.3 Full Integration

While the Drone side and Ground side are two separate processes, our final approach works to integrate them to work in unison.

5.3.1 Mission Life Cycle

The integration of both systems can properly be seen as we analyze the life cycle of a mission within the system. After initial connection between the drone and the ground station (Utilizing UDP broadcasting), we find that there exists 3 major phases of our project. Each phase, works to govern the specific behaviors of both the ground station pipeline and the drone's flight controller at any given moment. These phases dictate the flow of control authority and data from takeoff to landing.

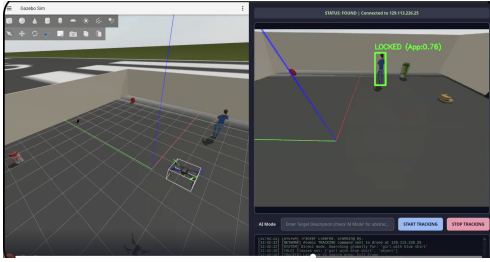


Figure 3: Gazebo and Gui Integration

Phase 1: Initial Searching: The first phase consists of the initialization of the user prompt. When the user presses the start button, two major things happen within the ground side, first an API call to the LLM is created. The ground station then defines a universal variable to monitor the current state of the life cycle. The current state variable is set to tracking, to acknowledge that the system is in search mode. This state, when changed, is then immediately sent to the drone side of the system utilizing the same UDP communication but on a different working port. When the drone receives this state command, the onboard relay node passes the instruction to the boustrophedon flight node. This triggers the autonomous sequence, causing the flight controller to arm, switch to **GUIDED** mode, ascend to its cruise altitude, and commence the 12×12 meter search pattern.

Phase 2: Dynamic Tracking The second phase consists of the successful detection and continuous tracking of the user's target. When the pipeline returns a positive match, two major things happen within the ground side: the transition of the detection pipeline, and the state update. When detected, the ground station updates the universal variable to monitor the current state of the program. Since the target has been located, we set the current state variable to **FOUND**, to acknowledge that the system is shifting from search mode to active tracking mode. This state, alongside the normalized pixel coordinates (bounding box) now continuously generated by YOLOv8m-World, is then immediately sent to the drone side of the system utilizing the active UDP connection. When the drone receives this state command and coordinate payload, the onboard relay node interrupts the boustrophedon flight node. This condition sends a signal that tells the autonomy loop to begin, causing the flight controller to abandon its predefined waypoints, apply an exponential moving average to the offset data, and incrementally steer the drone toward the target.

Phase 3: Mission Stoppage The third and final phase consists of the termination of the active mission and the safe recovery of the drone. When the user manually interrupts the operation by pressing the stop tracking on the UI, two major things happen within the ground side: the cessation of the vision pipeline, and the state reset. When triggered, the ground station updates the universal variable to **STOP** (Return-to-Launch), acknowledging that the system is shifting out of active tracking mode. It is then immediately sent to the drone side of the system through UDP connection. When the drone receives this state command, the onboard relay node intercepts the active tracking loop. This triggers the return sequence, causing the relay node to issue a service call to the MAVROS flight controller, forcing the drone to abandon its current approach vector, ascend to a safe return altitude, navigate back to its home coordinates, and autonomously land.

6 Results

To evaluate the system, we conducted mission trials in simulation across six prompt categories of varying semantic complexity. Each trial was scored on whether the drone locked onto the correct intended target. Trials were run under two configurations: *AI mode* (LLM + YOLOv8m-World) and *standard mode* (YOLOv8m-World only). For each prompt there was 5 trials each. Table 1 along with its coinciding figure summarize the correct detection rates across all testing prompts.

6.1 Simple Labels

For the generic prompt "*can*", both modes achieved a perfect detection rate of 5/5. When the target is unambiguous and has no competing instances of the same class

Table 1: Correct detections out of total trials per prompt and mode. A trial is marked correct only if the drone locked onto the intended target.

Prompt	AI Mode	Standard Mode	Notes
can	5/5	5/5	
red can	5/5	0/5	Standard: detected green can and red
red fruit	5/5	2/5	Standard: 3 RTL
yellow fruit	5/5	3/5	Standard: 2/5 wrong object
something to eat	4/5	0/5	Standard: false positives or RTL
book that boy is reading	5/5	0/5	Standard: all RTL

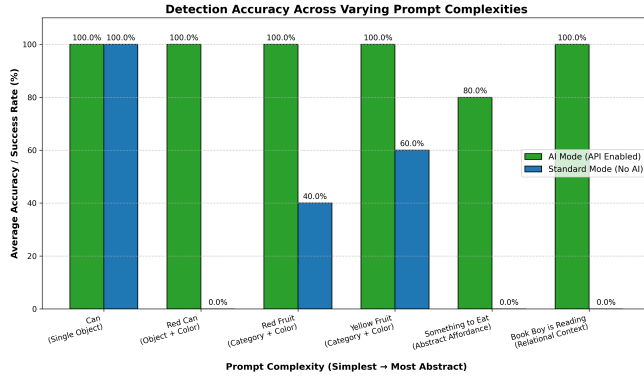


Figure 4: Visualized Data

existing in the scene, YOLOv8m-World is sufficient on its own and the LLM provides no additional benefit.

6.2 Color Disambiguation

The *“red can”* prompt exposed a fundamental limitation of YOLOv8m-World alone. With both a red and a green can present in the arena, standard mode failed to identify the correct target in all five trials, consistently locking onto both the red and the green cans. The word *“can”* dominated the match while the color constraint was ignored. AI mode corrected this entirely, achieving 5/5 accurate detections by using the LLM to reason about color before handing off to YOLOv8m-World.

6.3 Color-Plus-Category Queries

For *“red fruit”*, AI mode correctly identified the target in all five trials while standard mode succeeded only once in five attempts. For *“yellow fruit”*, AI mode was again perfect (5/5), while standard mode detected an object in all five trials but chose the wrong one in two of them. These results show that color-plus-category prompts sit at the edge of YOLOv8m-World’s capability: the category is often matched, but color-based selection between competing objects is unreliable without LLM arbitration.

6.4 Abstract and Intent-Level Descriptions

“Something to eat” represents a prompt with no direct object-class equivalent — the user is expressing intent rather than naming a specific item. AI mode handled this successfully in four of five trials, with the LLM identifying food-like objects in the scene and generating ap-

propriate YOLO class terms. Standard mode failed entirely: two trials ended in RTL without any detection, and the remaining three produced brief false positives that did not sustain a stable lock. Without the LLM, YOLOv8m-World has no mechanism to interpret intent-level language.

6.5 Relational Descriptions

“Book that boy is reading” requires the system to resolve a reference that spans two objects. AI mode succeeded in all five trials. Standard mode failed in all five trials with complete RTLs, as YOLOv8m-World cannot resolve relational or multi-object references from a text prompt alone.

6.6 Summary

Across all six prompt categories, AI mode achieved correct detections in 29 of 30 trials (95%). Standard mode succeeded in only 10 of 30 trials (30%), with failures concentrated entirely in prompts that required color disambiguation, abstract reasoning, or relational understanding. These results demonstrate that the incorporation of an LLM is not merely a marginal enhancement but a necessary component for any prompt beyond a simple, unambiguous object label.

7 Limitations

While we achieved a satisfiable result, we found that within our current iteration there exists a few forms of limitation. One issue involved our access to available hardware. We were able to properly achieve our pipeline using our personal computer and Gazebo, but the results were quite latent. In a real world setting, machines with greater computational capability would be necessary to avoid this latency issue. The biggest proponent of this issue was the use of an API call. Our computers (both companion and ground station) could not handle an LLM on board, so we were forced to make an API call that itself can take up to two seconds to receive a response. While we were able to solve this issue within simulation by adjusting speeds, in a real world setting this would be far from ideal. Our reliance on UDP network communication between devices in a real world scenario, is not always certain, so this reliance can be a major fault within the system. At the moment UDP communication was ideal for our testing case as we were certain our project would be tested within a network system, but we acknowledge that this may not always be the case and would have to be further evaluated.

8 Future Works

While we reached a successful conclusion to what we hoped to achieve within the semester, we acknowledge

that our technology can be further advanced. The current boustrophedon search pattern, while effective in controlled environments, represents a rigid, predefined approach to area coverage. Future iterations should work toward enabling the drone to intelligently navigate unknown, complex, and uncontrolled environments without relying on a predetermined path. One promising direction is the use of localization and mapping techniques such as SLAM, allowing the drone to build a real-time map of its surroundings and make informed navigational decisions based on the geometry of the environment. Traditionally UAV communication, involves radio waves rather than network communication. Our current method of network/UDP socket communication could be replaced with radio transmission signals in a fully deployed, production-ready iteration of this system. Additionally, the current system has been validated in simulation, transitioning to a fully physical deployment remains a critical milestone. Physical flight testing introduces environmental variables absent in simulation, including wind disturbance, GPS drift, uneven lighting conditions, and dynamic obstacles. Simulation uses an idealized sensor, whereas a physical deployment must contend with motion blur, lens distortion, and variable frame rates. Selecting and incorporating gimbal stabilization, would be necessary to maintain the quality of input frames required by both GPT and YOLOv8m-World. Rigorous field tests across a range of environments and target types would be essential to validate detection accuracy, latency, and overall system reliability before any operational deployment.

9 Conclusion

This project demonstrates that natural language task specification is a viable and effective paradigm for autonomous UAV detection. By combining LLM semantic reasoning with YOLOv8m-World open-vocabulary detection and a custom MAVLink communication interface, the system achieved a 95% correct detection rate across prompts ranging from simple labels to relational descriptions, compared to just 30% without LLM grounding. Although current limitations in hardware, physical testing space, and network reliability restrict the system to simulation, the architecture establishes a strong foundation for future deployment.

Acknowledgements

We would like to express our sincere gratitude to our advisor, Dr. Lu Qi, for his mentorship and continuous support throughout the development of this project. His guidance was instrumental in shaping our approach to this project. We also extend our deepest appreciation to the Multiple Autonomous Robotic Systems(MARS) Lab for providing the resources and collaborative environment that made this work possible. Finally, a special thank you to Javier Berrecil for their hands-on technical

assistance, insightful feedback, and dedication to helping us troubleshoot complex challenges along the way.

References

- [1] Ardupilot. <https://ardupilot.org>.
- [2] Gazebo. https://gazebo.org/docs/harmonic/install_ubuntu/.
- [3] Mavlink. <https://mavlink.io/en/>.
- [4] Robot operating system. <https://docs.ros.org/en/jazzy/index.html>.
- [5] Senior project github repository. <https://github.com/isabellawtu09/Senior-Project-Autonomous-Drone/tree/main>.
- [6] G. Chen, X. Yu, N. Ling, and L. Zhong. Typefly: Flying drones with large language model. *arXiv*, 2023.
- [7] F. Ciccone and A. Ceruti. Real-time search and rescue with drones: A deep learning approach for small-object detection based on yolo. *Drones*, 9(8), 2025.
- [8] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics yolov8, 2023.
- [9] A. Raman, C. Merrill, A. George, and A. B. Farimani. Llm-drone: Aerial additive manufacturing with drones planned using large language models. *arXiv*, 2025.
- [10] R. Sinha, A. Elhafi, C. Agia, M. Foutter, E. Schmerling, and M. Pavone. Real-time anomaly detection and reactive planning with large language models. *arXiv*, 2024.
- [11] W. Wu, C. Li, X. Wang, B. Luo, and Q. Liu. Large language model guided progressive feature alignment for multimodal uav object detection. *arXiv*, 2025.